

第 10 课

多 GPU 第 2 部分

Multi GPU Part 2

第 10 课 · 多 GPU 第 2 部分

多 GPU

第 2 部分 Prateek Shukla

Slurm

SLURM 是一个开源的工作负载管理器和作业调度器 (job scheduler)，为基于 Linux 的高性能计算集群而设计。

它分配 GPU、CPU 和内存。它知道你的作业在哪里运行，但不一定知道你的应用程序如何跨这些节点与自己通信。

在早些年，mpirun 会用 SSH 连接到每个节点，检查主机名并交换密钥。在大型集群（例如 64+ 个节点）上，这种"握手" (handshake) 可能需要几分钟

使用 `srun --mpi=pmix`，Slurm 在所有节点上同时启动这些进程。

PMIx (Process Management Interface for Exascale)

当 Slurm 启动你的 1,000 个进程。此时 rank 5 (进程 #5) 知道自己活着，但它不知道 rank 0 的 IP 地址，也不知道如何与 rank 999 通信。它们是一座座孤岛。

在高强度训练开始之前，进程之间需要交换技术细节 (IP 地址、GPU 句柄)。PMIx 提供了一个临时数据库。

PMIx_Put: 进程把自己的元数据 (例如主机 IP、端口、CUDA IPC 句柄) 推送到本地 PMIx 服务器。

PMIx_Commit: 把本地数据推送到全局命名空间。

PMIx_Get: 其他进程查询这些数据来发现自己的对等方。

为什么 SLURM 和 PMIx 搭配使用很重要

单靠 Slurm 可以处理资源分配（节点、GPU、CPU）和任务放置，但它历史上依赖的旧版 PMI 在超过约 10k 个 GPU 之后扩展性并不好。

PMIx 是现代的、面向 exascale 的进程管理接口（取代 PMI-1/PMI-2）。

Slurm + PMIx 的集成带来：

在许多情况下无需 mpirun 即可直接启动进程。

在大规模下高效交换作业信息（rank、节点列表、端点）。

通过直接的 NCCL-PMIx 插件与 NCCL 紧密集成。

SLURM 脚本

它是一个简单的文本文件（通常以 .sh 或 .sbatch 结尾），告诉 Slurm 两件事：

你需要什么资源（时长、GPU、CPU、内存）。

拿到这些资源之后你想做什么（运行 Python、编译代码等）。

写好脚本之后，我们需要用 `sbatch script.sh` 来启动

脚本启动后，Slurm 会登录计算节点，设置好环境，并运行脚本中列出的命令。

术语

作业：作业是 Slurm 中最高层的工作单元。它代表资源分配。当你运行 sbatch 或 salloc 时，你就是在创建一个作业。

任务：任务是运行中的应用程序的一个单独进程。它是每个节点/GPU 上运行的进程数量。

Rank：rank 就是任务的 ID；除非特别说明，它指的是全局 rank，即整个作业中的 rank。

Local rank：这是任务在特定节点内的唯一 ID。范围从 0 到单个节点上运行的任务数

命名空间：对作业内的每个 rank 而言，这就是 jobID。对作业中的每个 rank 都是唯一的

设备隔离

如果你设置 slurm 让每个任务使用单个 GPU，那么 slurm 不是礼貌地请求你的程序只用一个 GPU，而是在操作系统层面用两种机制强制执行：

CUDA_VISIBLE_DEVICES：Slurm 在进程内部设置这个环境变量。它让 CUDA 看不到此列表之外的任何其他设备。适用于单节点

Slurm 使用 Linux Control Groups (cgroups) 特性，具体来说是设备白名单 (device allowlist)，它为任务 0 创建了一个"沙盒"。

这意味着，如果你在调试且每个 GPU 上运行着不同的任务，那么查看日志时，每一条错误都会指向 GPU 0，即使它实际上并不是 GPU 0

第 10 课 · 多 GPU 第 2 部分

一个脚本示例

SLURM 脚本如何为 kernel 启动 PMIx

当你运行 `srun --mpi=pmix ./my_cuda_kernel_app` 时，Slurm 不是盲目地 fork 进程。它使用 PMIx 来编排启动过程。

之后我们必须用以下步骤初始化 PMIx：

1. 第 1 步：连接 slurm 守护进程 (daemon)
2. 第 2 步：初始化 PMIx，获取命名空间和全局 rank
3. 第 3 步：获取 local rank
4. 第 4 步：用 Local Rank 选择特定的一个或多个 GPU
5. 第 5 步：结束进程

第 10 课 · 多 GPU 第 2 部分

初始化 PMIx 并把进程连接到 slurm

获取身份（全局 rank 和 local rank）

Slurm 通常通过设置 `CUDA_VISIBLE_DEVICES` 来限制每个任务的 GPU 可见性，

因此你的进程看到的是一组经过过滤、重新编号的 GPU，从设备 0 开始。

PMIx 提供节点本地 rank (`PMIX_LOCAL_RANK`)，这是把任务分布到该节点被指派的 GPU 上时自然而然要使用的索引。

用 `cudaGetDeviceCount` 了解过滤后有多少设备可见；如果需要原始物理索引，可以选择解析 `CUDA_VISIBLE_DEVICES`。

要做更深入的调试，可查询 PCI 总线 ID (`cudaDeviceGetPCIBusId`)，这样你就能把"可见设备 0"与 Slurm 实际分配的那块硬件 GPU 对应起来。

绑定到本地 GPU

之后启动 kernel，等它们完成后，在 host 函数里使用 `PMIx_Finalize(NULL, 0)`; 告诉 SLURM 我们已经完成

那么 NCCL 是什么

NVIDIA Collective Communications Library。它是一个以 GPU 为核心的库，让 GPU 之间的集合通信变得快速（而且相对无痛）

在分布式训练中，GPU 需要不断交换张量（梯度/参数）。NCCL 为此提供了高度优化的原语，例如 Allreduce、Allgather、ReduceScatter 等。

它在 CUDA 流 + 指针这一层面集成：你把设备指针和一个 `cudaStream_t` 交给 NCCL，NCCL 在该流上调度它自己的 GPU 工作（kernel + memcpy + 网络操作），因此它通过正常的 CUDA 流顺序与你的 kernel 组合在一起。

当你执行一个 allReduce 操作时，CPU 只是把一个 NCCL kernel 发射到 GPU 流上。

用 NCCL 建立通信

在任何通信发生之前，NCCL 需要一种办法让所有进程找到一个公共会合点。这个会合点由一段数据表示，称为 NCCL Unique ID。

只有一个进程能生成这个 ID，而每一个想加入这个组的进程都必须持有完全相同的这个 ID。

rank 0 让 NCCL 创建一个新的 Unique ID，拿到它并放进 PMIx 键值存储（Key-Value Store，KVS），让其他进程可以读取。

rank 0 让 PMIx commit 这些数据，确保它真正被推送到服务器并对整个网络可见。一次 fence 保证所有人都能读到数据，然后跟随者（followers）检查键值存储并调用 NCCL 初始化函数

ncclGetUniqueId

刚开始时，GPU 不知道自己周围有任何其他 GPU，也不知道如何用 nvlink/infiniband 到达它们。

为了与其他 GPU 通信，NCCL 需要构建一个"通信器"

NCCL 调用 ncclGetUniqueId 创建 ncclUniqueId 结构体，并登记第一块 GPU 的 IP 地址。随后每个 GPU 都进来，在这个结构体里登记自己的 IP，以便相互通信

我们将看到如何用 PMIx 来做到这一点

第 10 课 · 多 GPU 第 2 部分

用法

NCCL 非常快，但启动时它并不知道其他 GPU 在哪里。

不同的进程运行在不同的 GPU 上。它们拥有各自独立的内存。进程 B 看不到进程 A 内部的变量。

我们需要确保参与者 commit 的数据被"收集"起来，并按照作用域规则（例如 PMIX_GLOBAL）变得可用。我们创建一个 fence，在所有人都到达之前没有人返回。

然后每个进程都需要找到那个键值存储；为此我们必须指定它归谁所有。做法是使用这些进程的 nspace 并把 rank 设为 0。

我们用 PMIx_Get 取回通过 PMIx_Put 发布的那个键；这个键不承担任何安全或验证职能，它纯粹是用来给数据寻址的。

完成之后，我们把 NCCL unique ID 的字节从 PMIx 的临时缓冲区复制到我们自己的局部变量 ncclUniqueld id 中。

ncclCommonInitRank

这是建立阶段最昂贵、也最复杂的函数。ID 就是在这里映射到硬件连接的；我们把 nrank 个 rank 的 ID 登记到 GPU0 上

每个 rank 都查看 ncclUniqueId 结构体，并从中提取 rank 0 的 IP:Port。

每个 rank 创建一个标准 TCP socket 并连接到 rank 0。

rank 0 一直等到恰好收到 nrank 个连接。

这是一个屏障（barrier）。如果 rank 799 很慢，所有人都得在这里等。

TCP 连接建立之后，它们还不会发送数据，而是先发送关于各自 rank、连接类型、设备等的元数据。

接下来的步骤

rank 0 扮演架构师。它分析全局图，为 AllReduce 这类操作找出带宽最高、延迟最低的路径。

节点内通信它优先使用 NVSwitch；节点间跳转它选择特定的 InfiniBand NIC；至于通信模式，它决定构建偏延迟优化的环形结构，还是偏带宽优化的树形结构。

rank 0 把具体的"路由表"（发给谁、从谁那里收）发回给每个 rank。在单个节点内，各 rank 相互映射对方的内存。配置 NVSwitch，允许 GPU 到 GPU 的直接内存访问

各 rank 识别自己在其他节点上的配对对等方，并做 RDMA 握手，确保无需 CPU 介入就能直接写入彼此的内存缓冲区。

ncclCommDestroy

这就是拆除我们先前创建的那些硬件路径的方法

它把通信器对象标记为无效，不能用于后续的 kernel 发射。如果 GPU 当前正在执行某个 NCCL kernel（例如 CUDA 流内的一个 AllReduce），ncclCommDestroy 不会立即把内存从它脚下抽走。它依赖内部引用计数

Init 期间在 HBM 中分配的 4MB-8MB 暂存缓冲区会被释放。

用于 PCIe 传输的 CPU RAM 中的暂存区域会被释放

NvLink 映射和 Infiniband Queue Pair 会被销毁

NCCL 集合通信原语

在 H100 集群上，我们不按 send 和 receive 来思考，而是按跨 NVLink fabric 的数据操作模式来思考。以下是 6 大原语 Broadcast - 一个 GPU 把自己的数据复制到所有 GPU。Reduce - 所有 GPU 合并数据，结果落在一个 GPU 上。AllReduce - 所有 GPU 得到所有人数据的归约和。AllGather - GPU 从碎片开始，最后得到完整合并后的缓冲区。ReduceScatter - 先归约，再切分输出，让每个 GPU 拿到不同的块。All-to-All - 每个 GPU 给其他每个 GPU 发送一份，并收回属于自己的部分

AI 中 4 种重要的并行类型

在这些多 GPU 系统上做 AI 应用时，会遇到 4 种并行类型

1. 数据并行 2. 张量并行 3. 流水线并行 4. 专家并行

下面我们逐一详细讨论

数据并行

数据并行是深度学习中最常用的分布式训练策略。它把输入数据分发到多个 GPU 上，同时在每个设备上复制模型本身，从而让你更快地训练模型

每个 GPU 上都放一份完整模型的副本。全局的训练数据批次被切分成更小的"迷你批次" (mini-batch)。每个 GPU 拿到属于自己的一份独特数据切片。每个 GPU 对自己那份数据执行前向和反向传播。在优化器更新模型权重之前，系统必须确保每一份模型副本保持一致。来自所有 GPU 的梯度被聚合（通常取平均）。

对数据并行来说，严格讲你主要依赖 AllReduce，但 Broadcast 对初始化必不可少

涉及的 NCCL 操作

AllReduce 是数据并行训练中最重要的单一操作。它发生在每次反向传播的末尾。GPU 1 有梯度 g_1 ，GPU 2 有梯度 g_2 ，等等。我们需要每个 GPU 都拿到平均梯度 $(1/N)g_i$ 。

AllReduce 把来自所有 GPU 的向量求和，并把结果分发回所有 GPU。

Broadcast 通常用在训练的最开始，或从 checkpoint 恢复时。在权重初始化阶段，rank 0 初始化参数并把它们 Broadcast 给所有其他 rank。这保证了在第 0 步时，所有副本在数学上完全一致

张量并行

如果你试图加载一个对单个 GPU 来说过大的模型，程序会在计算还没开始时就直接崩溃。当权重矩阵大到无法在单个 GPU 上做矩阵乘法时，就要用张量并行

如果你是在训练模型（而不只是运行它），内存需求会变成三倍甚至四倍。你不只要存权重，还需要存梯度、优化器状态和激活值。

我们在这里的做法是：把矩阵切成若干块，分发到不同的 GPU 上计算，等计算完成后再把所有结果拼到一起。

序列并行

在标准张量并行中，我们把沉重的矩阵乘法（Linear 层）拆分到各个 GPU 上。但我们不拆分发生在 Linear 层之间的那些操作，具体是：LayerNorm、Dropout、GeLU/SiLU 激活

在标准 TP 中，（Linear 层之后）AllReduce 的输出是一个被复制的张量。这意味着如果你有 8 个 GPU，每个 GPU 都要存一份完全相同的完整激活矩阵副本（尺寸：[Sequence Length, Hidden Dimension]），就只是为了执行 LayerNorm 或 Dropout。

LayerNorm 作用于单个 token 的 Hidden Dimension，它在序列维度（Sequence Dimension）上是独立的。因此我们不需要在每个 GPU 上复制完整序列。针对这些操作，我们可以把序列分区（partition）到各个 GPU 上。

在 TP 中拆解 allreduce

标准 TP 用 AllReduce 来同步矩阵乘法的输出。从数学上讲，AllReduce 实际上是两个原语操作的组合：

$\text{AllReduce} = \text{ReduceScatter} + \text{AllGather}$

使用 SP 时，SP 不是把 AllReduce 当作原子操作完成（NCCL 融合），而是把 LayerNorm/Dropout 操作注入到通信循环内部

我们先执行 ReduceScatter，在那里停下，在分片数据上做 LayerNorm，只在下一步矩阵乘法绝对需要完整数据时才做 AllGather。

TP 中涉及的 NCCL 操作

Broadcast：在按列（Column-wise）分片中用于把完整的输入矩阵复制给每个 worker。All-Gather：在按列分片中用于合并乘法之后的结果；在序列并行的前向传播中用于合并分片的序列块。All-Reduce：在按行（Row-wise）分片中用于对不同 worker 的结果求和得到最终结果。在一个标准 Transformer 块中，张量并行意味着每个 transformer 块有两次 all-reduce（一次用于 Attention 块，一次用于前馈（Feedforward）块）。Reduce-Scatter：在序列并行中用于在沿序列维度 scatter 的同时对梯度做归约。在 TP 中，Reducescatter 用于列向 linear 操作反向传播时的梯度。

流水线并行

如果说张量并行是把单个层横向切片，那么流水线并行就是把模型纵向切片（拆分层堆栈）。

把模型的各层拆到不同 GPU 上之后，每个 GPU 只需要存它那一块层的参数和优化器状态。

如果你只是把一个大批次直接送进流水线，大多数 GPU 都会闲置着等待轮到自己。流水线并行把一大批数据拆成很小的微批次（micro-batch）。GPU 1 一完成第一个微批次，就把它传给 GPU 2，并立刻开始处理第二个微批次

由于 PP 只需要流水线中"邻居"之间的通信，它只使用点对点的 send/receive 操作。

专家并行

标准的"稠密" (dense) 模型对每个输入会用到每一个参数；与此不同，MoE 模型对每个输入只激活一小部分参数 (experts)。

这对成本有利。你可以把参数量提高 100 倍 (增加更多 experts)，但只要每个 token 只选 top-2 experts，计算成本仍然相对较低。

模型会学到某些"experts"擅长编程，而另一些擅长创意写作。专家并行确保当一个编程 token 进来时，它被专门路由到持有"编程 expert"的那个设备。

专家并行如何工作

一个小网络决定这个 token 需要 expert A 还是 expert B 等等。

experts 分布在各个 GPU 上。GPU 1 持有 Expert A 和 B，GPU 2 持有 Expert C 和 D，GPU 3 持有 Expert E 和 F，以此类推

如果 GPU 1 上的一个 token 需要 Expert D（它在 GPU 2 上），就必须通过网络把它发送到 GPU 2。最终每个 GPU 都需要给其他每个 GPU 发数据。这被称为"all to all"通信。

像 DeepEP 这样的专用库正是专精于这类场景：它们先把要发送的数据打包，高效地跨节点间连接发送，然后再用更快的 nvlink 连接在本地快速分发。

NCCL 操作的一般格式

sendbuff、recvbuff、count、datatype 告诉 GPU 要移动多少元素以及如何解释它们。

op 告诉 GPU 要执行哪种数学运算

comm 是通信句柄，保存节点和集群的所有元数据

stream 用来指定异步行为

Broadcast

NCCL 中的 Broadcast 操作是一种集合通信原语：一个"根（Root）" GPU 把一个数据张量发送给通信器中的所有其他 GPU。

一般来说它用 ring/tree 操作实现，但在 H100 DGX 上，根 GPU 只把数据包向 NVSwitch 发送一次，NVSwitch 自己在物理上把数据包同时复制到其余 7 个端口（单节点内）。

由于复制由交换机完成，向 8 个 GPU broadcast 所需的时间与只发给 1 个 GPU 大致相同。

ncclBroadcast

root: 持有我们要 broadcast 的数据的那个 GPU 的 rank sendbuff: 我们要 broadcast 的数据缓冲区的位置 recvbuff: 所有 GPU (包括根) 上的目的指针 (如果两者相同, 则启用原地 (in-place) reduction)。Datatype: 如果你传入 ncclFloat8e4m3 或 ncclFloat8e5m2, NCCL 会切换到使用 Hopper 专属的 intrinsic 指令 comm: 通信器对象 stream: CUDA 流

操作的数学

设 $P=\{0,1,\dots,N-1\}$ 为通信器中 N 个进程 (GPU) 的集合。设 $V(i)$ 表示进程 i 持有的数据向量。

Reduction

Reduction 从每个参与的 GPU 拿一个数据数组，用一个数学操作符（例如 sum 或 max）把它们合并，并把合并后的单一结果存到某个特定 GPU（或所有 GPU）上。

在以前的系统中，GPU 必须互相传递数据（像救火队传水桶一样）才能完成求和。在 DGX H100 上，NVSwitch 芯片自己执行这些数学运算，把工作从 GPU 上卸载下来。

一般性的 reduction（ncclReduce）通常使用 NVLSTREE（NCCL 2.18+ 引入）来处理把数据归约到单个根的特定流程，尤其是在跨多个节点扩展时

ncclReduce

当数据的消费者是集中的（通常是 rank 0），而其他 rank 不需要立刻拿到结果才能继续时，你用 ncclReduce。

它一般用在推理的最后一层：只有 rank 0 需要完整的 logits 来执行 argmax 或 top-k 采样。

count 是每个 GPU 提供的元素数量。根收到的恰好是 count 个元素。

设 P 为处理器（GPU）的数量，索引从 0 到 $P-1$ 。每个处理器 p

持有一个大小为 N 的输入向量 V_p : $V_p=[v_{p,0},v_{p,1},...,v_{p,N-1}]$

AllReduce

AllReduce 是 AI 模型训练中最重要的操作之一

合并来自 N 个 GPU 的数据 -> 得到一个全局结果 -> 分发给 N 个 GPU

全部 8 个 GPU 从各自的 HBM3 内存读取本地梯度

不是把数据发给某个对等 GPU，而是全部 8 个 GPU 同时把各自的数据推上 NVLink 通道，目标是 NVSwitch 芯片

数据到达交换机，reduction 在"传输途中" (in flight) 发生，交换机立刻把这个结果同时多播回所有 8 个 GPU。

结果直接落入所有 GPU 上 HBM3 中的目标缓冲区。

ncclAllReduce

这是深度学习用例中最流行、最重要的操作之一

当每个参与者既提供数据、又需要最终归约结果时——例如跨 worker 的梯度或指标聚合——AllReduce 就是正确的原语

ncclRedOp_t

这是归约所用的数学操作（ncclSum、ncclProd、ncclMin、ncclMax、ncclAvg）

ncclSum 是唯一一个在 H100 上对每条硬件路径都完全优化的操作符。如果你用它，NCCL 会把所有操作卸载到 nvswitch

ncclMin 和 ncclMax 也可以被硬件卸载（fp8）

ncclProd 会把所有操作卸载到 tensor core，因为 nvswitch 做不了浮点乘法

ncclAvg 分两部分完成：求和部分在 nvswitch 上完成，除法部分在 GPU 上完成

AllReduce 的数学

设有 P 个进程。每个进程 k 持有一个大小为 N 的向量 V_k 。设 $V_k[i]$ 表示进程 k 上向量的第 i 个元素。

AllReduce 的目标是让每个进程 k 最终都得到一个结果向量 R ，其中第 i 个元素是该元素在所有进程上的和（或其他满足结合律的操作符的结果）

它被用来对梯度求和： V_k 是 GPU k 计算出的梯度向量， R 是用于更新模型的总梯度。

ReduceScatter

ReduceScatter 是这样一种操作：每个 GPU 一开始都持有一个完整的梯度缓冲区，目标是跨所有 GPU 对这些缓冲区求和，然后把结果 scatter 开去，让每个 GPU 最终只持有最终求和向量的一个互不相同的"切片" 输入：每个 GPU 有一个向量 [A,B,C,D]。数学：跨所有 GPU 的逐元素求和 输出：GPU 0 持有 Sum(Part A) GPU 1 持有 Sum(Part B) GPU 2 持有 Sum(Part C) GPU 3 持有 Sum(Part D)

ncclReduceScatter

size_t recvcount: 这是输出缓冲区的元素数量，不是整个输入缓冲区的。它等于梯度缓冲区中的总元素数 / GPU 数量 被归约的总元素数 = $\text{recvcount} * \text{n ranks}$

如果你的总向量大小是 100 而有 3 个 GPU: 100 无法被 3 整除 (33.33...)。NCCL 不支持 GPU 0 拿 34、GPU 1 拿 33 这种"参差不齐" (jagged) 的数组。因此你必须把数据填充 (pad) 到 GPU 数量的下一个倍数

ReduceScatter 的数学

AllGather

AllGather 是这样一种操作：每个 GPU 一开始持有梯度的不同片段，变换完成之后，每个 GPU 都拿到完整数据的一份完整副本

全部 8 个 GPU 同时把自己的梯度推入 NVSwitch fabric

NVSwitch 把数据合并起来，并把结果路由到所有 GPU

总线总利用率被最大化。你能更接近理论上的 900 GB/s 聚合吞吐，因为你不必等一个环轮转一圈

sendcount 是该 rank 提供的元素数量（即其本地

切片）的大小。每个 rank 最终都会在 recvbuff 中拿到所有 rank 的切片。

第 10 课 · 多 GPU 第 2 部分

从数学上讲

All to All

节点上每个 GPU 都持有发给其他每个 GPU 的数据，并把数据传输给相应的 GPU

它的工作方式是：每个 GPU 从每个 GPU 接收属于自己的那份数据，并把属于每个 GPU 的数据发送出去

AllReduce 常常使用复杂的 Ring 或 Tree 算法以及硬件卸载（SHARP）；与之不同，本系统上的 AllToAll 在物理上更简单，但对带宽的消耗很大。

假设通信器中有 $\text{nranks} = N$ 个 GPU。

你有一个在逻辑上切分成 N 个块的发送缓冲区：块 j 是你想发送给 rank j 的数据。

all-to-all 之后，每个 rank 有一个切分成 N 个块的接收缓冲区：块 i 包含 rank i 发给你的内容。

所以这就像在做 $N \times (N-1)$ 次点对点传输（再加上自己给自己的那份），但它们作为一个集合通信操作被协调和优化。

MoE 的问题

对 MoE 模型通常要避免使用 NCCL 的标准 AllToAll，因为它为静态的、大批量的同步通信设计的，而 MoE 路由天然动态、稀疏且对延迟敏感的。

NCCL All to all 的主要问题包括：NCCL 由 CPU 驱动（Host API）、NCCL 使用静态的数据大小、NCCL 要求数据位于连续的块中，以及这条指令本身的阻塞性质。

像 DeepEP 这样的专用库（以及底层的 NVSHMEM 原语）更受青睐，因为它们允许由设备发起的、细粒度的数据移动，能处理 expert 路由的不规则流量模式，而不会让 GPU 停摆。

ncclGroupStart ncclGroupEnd

在分布式系统中，很多时候是一个 CPU 线程管理多个 GPU。当 CPU 发射一条 NCCL 指令时，如果它是阻塞调用，CPU 会被 NCCL 阻塞。大多数 NCCL 集合通信调用（例如 `ncclAllReduce`）对 CPU 而言是异步的，更容易造成阻塞的往往是通信器初始化（`ncclCommInitRank`），

如果一个 host 线程要为多个 GPU 发出 NCCL 调用，逐个发射会制造出部分状态：一些参与者已经开始某个操作，而另一些还没被发射，这可能导致挂起

`ncclGroupStart()/ncclGroupEnd()` 让你把一组 NCCL 调用打包成批：NCCL 在 group 期间收集这些调用，然后在 `ncclGroupEnd()` 时一起提交。这避免了由部分发射的 NCCL 工作所引发的问题。

ncclSend

它是一个非阻塞操作，用于把数据从一个特定 GPU（发送方）发送到另一个特定 GPU（接收方）。它几乎总是与目标设备上对应的 `ncclRecv` 配对使用。

`peer`：目的 GPU 的 rank（ID）。

NCCL 会在两个 GPU 之间找出可用的最快物理路径，可能是 NVLink、PCIe 或 IB RDMA

ncclRecv

告诉某个特定 GPU 在自己的内存中分配空间，并等待来自指定"对等方 (peer)"的数据。它是异步的，这意味着命令在 CUDA 流上入队之后，CPU 立即恢复控制 peer：发送数据的那个 GPU 的 rank (ID)。一个 ncclRecv 必须在源 GPU 上有匹配的 ncclSend。如果你在做多个 P2P 传输，它们必须以相互兼容的顺序在不同 GPU 上入队，以避免循环依赖

防止所有 GPU 发射同一条指令

第 10 课 · 多 GPU 第 2 部分

大概就到这里